

The Option-Critic Architecture

Pierre-Luc Bacon and Jean Harb and Doina Precup

Reasoning and Learning Lab, School of Computer Science
McGill University
{pbacon, jharb, dprecup}@cs.mcgill.ca

Abstract

Temporal abstraction is key to scaling up learning and planning in reinforcement learning. While planning with temporally extended actions is well understood, creating such abstractions autonomously from data has remained challenging. We tackle this problem in the framework of options [Sutton, Precup & Singh, 1999; Precup, 2000]. We derive policy gradient theorems for options and propose a new *option-critic* architecture capable of learning both the internal policies and the termination conditions of options, in tandem with the policy over options, and without the need to provide any additional rewards or subgoals. Experimental results in both discrete and continuous environments showcase the flexibility and efficiency of the framework.

Introduction

Temporal abstraction allows representing knowledge about courses of action that take place at different time scales. In reinforcement learning, *options* (Sutton, Precup, and Singh 1999; Precup 2000) provide a framework for defining such courses of action and for seamlessly learning and planning with them. *Discovering* temporal abstractions autonomously has been the subject of extensive research efforts in the last 15 years (McGovern and Barto 2001; Stolle and Precup 2002; Menache, Mannor, and Shimkin 2002; Şimşek and Barto 2009; Silver and Ciosek 2012), but approaches that can be used naturally with continuous state and/or action spaces have only recently started to become feasible (Konidaris et al. 2011; Niekum 2013; Mann, Mannor, and Precup 2015; Mankowitz, Mann, and Mannor 2016; Kulkarni et al. 2016; Vezhnevets et al. 2016; Daniel et al. 2016).

The majority of the existing work has focused on finding *subgoals* (useful states that an agent should reach) and subsequently learning policies to achieve them. This idea has led to interesting methods but ones which are also difficult to scale up given their “combinatorial” flavor. Additionally, learning policies associated with subgoals can be expensive in terms of data and computation time; in the worst case, it can be as expensive as solving the entire task.

We present an alternative view, which blurs the line between the problem of *discovering options* from that of *learning options*. Based on the policy gradient theorem (Sutton et al. 2000), we derive new results which enable a gradual

learning process of the intra-option policies and termination functions, simultaneously with the policy over them. This approach works naturally with both linear and non-linear function approximators, under discrete or continuous state and action spaces. Existing methods for learning options are considerably slower when learning from a single task: much of the benefit comes from re-using the learned options in similar tasks. In contrast, we show that our approach is capable of successfully learning options within a single task without incurring any slowdown and while still providing benefits for transfer learning.

We start by reviewing background related to the two main ingredients of our work: policy gradient methods and options. We then describe the core ideas of our approach: the intra-option policy and termination gradient theorems. Additional technical details are included in the appendix. We present experimental results showing that our approach learns meaningful temporally extended behaviors in an effective manner. As opposed to other methods, we only need to specify the number of desired options; it is not necessary to have subgoals, extra rewards, demonstrations, multiple problems or any other special accommodations (however, the approach can take advantage of pseudo-reward functions if desired). To our knowledge, this is the first end-to-end approach for learning options that scales to very large domains at comparable efficiency.

Preliminaries and Notation

A Markov Decision Process consists of a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a transition function $P : \mathcal{S} \times \mathcal{A} \rightarrow (\mathcal{S} \rightarrow [0, 1])$ and a reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. For convenience, we develop our ideas assuming discrete state and action sets. However, our results extend to continuous spaces using usual measure-theoretic assumptions (some of our empirical results are in continuous tasks). A (Markovian stationary) *policy* is a probability distribution over actions conditioned on states, $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$. In discounted problems, the value function of a policy π is defined as the expected return: $V_\pi(s) = \mathbb{E}_\pi [\sum_{t=0}^\infty \gamma^t r_{t+1} \mid s_0 = s]$ and its action-value function as $Q_\pi(s, a) = \mathbb{E}_\pi [\sum_{t=0}^\infty \gamma^t r_{t+1} \mid s_0 = s, a_0 = a]$, where $\gamma \in [0, 1)$ is the *discount factor*. A policy π is *greedy* with respect to a given action-value function Q if $\pi(s, a) > 0$ iff $a = \operatorname{argmax}_{a'} Q(s, a')$. In a discrete MDP, there is at least one optimal policy which is greedy with re-

选项-评论家架构

Pierre-LucBacon 和Jean Harb 以及DoinaPrecup推理

与学习实验室，麦吉尔大学计算机科学学院{pbacon, jharb, dprecup}@cs.mcgill.ca

摘要

时间抽象是扩展强化学习中学习与规划能力的关键。虽然对时间扩展动作的规划已被充分理解，但从数据中自主创建此类抽象仍然具有挑战性。我们在选项框架中解决这一问题 [萨顿、普雷卡普和辛格，1999；普雷卡普，2000]。我们推导了选项的策略梯度定理，并提出了一种新的选项评论家架构，该架构能够同时学习选项的内部策略和终止条件，以及与选项之上的策略协同工作，且无需提供任何额外的奖励或子目标。在离散和连续环境中的实验结果展示了该框架的灵活性和效率。

引言

时间抽象允许表示在不同时间尺度上发生的行动过程的知识。在强化学习中，选项（Sutton、Precup和Singh 1999；Precup 2000）为定义此类行动过程以及无缝学习和规划提供了框架。发现时间抽象的自主性在过去15年中一直是广泛研究工作的主题（McGovern和Barto 2001；Stolle和Precup 2002；Menache、Mannor和Shimkin 2002；S. ims.ek和Barto 2009；Silver 和 Ciosek 2012），但能够自然用于连续状态和/或动作空间的方法直到最近才开始变得可行（Konidaris等人，2011；Niekum 2013；Mann、Mannor和Precup 2015；Mankowitz、Mann和Mannor，2016；Kulkarni等人，2016；Vezhnevets等人，2016；Danielet al.2016）。

现有的大多数工作都集中在寻找子目标（智能体应到达的有用状态），并随后学习实现这些目标的策略。这一思路催生了一些有趣的方法，但由于其“组合式”特性，这些方法也难以扩展。此外，与子目标相关的策略学习在数据和计算时间方面可能代价高昂；在最坏的情况下，其成本可能与解决整个任务相当。

我们提出了一种替代视角，模糊了从问题中发现选项与学习选项之间的界限。基于策略梯度定理（Sutton等人2000），我们推导出新的结果，使得能够逐步

学习选项内策略和终止函数的过程，同时学习其上的策略。该方法在离散或连续状态和动作空间下，自然适用于线性和非线性函数逼近器。现有的学习选项方法在从单一任务中学习时速度明显较慢：其大部分收益来自于在相似任务中重用已学习的选项。相比之下，我们证明我们的方法能够在单一任务中成功学习选项，而不会产生任何速度下降，同时仍然为迁移学习提供益处。

我们首先回顾与本研究两大核心要素相关的背景知识：策略梯度方法和选项。随后，我们描述本方法的核心思想：选项内策略和终止梯度定理。其他技术细节包含在附录中。我们展示的实验结果表明，本方法能够以有效的方式学习有意义的、时间上扩展的行为。与其他方法不同，我们只需指定所需选项的数量；无需子目标、额外奖励、演示、多个问题或任何其他特殊设置（不过，如有需要，本方法也可以利用伪奖励函数）。据我们所知，这是第一个能够以相当效率扩展到超大规模领域的端到端选项学习方法。

预备知识与符号说明

马尔可夫决策过程由一组状态 $\mathcal{S}$ 、一组动作 $\mathcal{A}$ 、转移函数 $P : \mathcal{S} \times \mathcal{A} \rightarrow (\mathcal{S} \rightarrow [0, 1])$ 和奖励函数 $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 组成。为方便起见，我们假设状态和动作集合是离散的来阐述我们的思想。然而，我们的结果在通常测度论假设下可扩展到连续空间（我们的一些实证结果来自连续任务）。一个（马尔可夫平稳）策略是在给定状态下动作上的概率分布， $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ 。在折扣问题中，策略 π 的值函数定义为期望回报： $V_\pi(s) = \mathbb{E}_\pi [\sum_{t=0}^\infty \gamma^t r_{t+1} \mid s_0 = s]$ ，其动作值函数为 $Q_\pi(s, a) = \mathbb{E}_\pi [\sum_{t=0}^\infty \gamma^t r_{t+1} \mid s_0 = s, a_0 = a]$ ，其中 $\gamma \in [0, 1)$ 是折扣因子。策略 π 相对于给定动作值函数 Q 是贪心的，如果 $\pi(s, a) > 0$ 当且仅当 $a = \operatorname{argmax}_{a'} Q(s, a')$ 。在离散马尔可夫决策过程中，至少存在一个最优策略，该策略相对于其自身的动作值函数是贪心的。

spect to its own action-value function.

Policy gradient methods (Sutton et al. 2000; Konda and Tsitsiklis 2000) address the problem of finding a good policy by performing stochastic gradient descent to optimize a performance objective over a given family of parametrized stochastic policies, π_θ . The policy gradient theorem (Sutton et al. 2000) provides expressions for the gradient of the average reward and discounted reward objectives with respect to θ . In the discounted setting, the objective is defined with respect to a designated start state (or distribution) s_0 : $\rho(\theta, s_0) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0]$. The policy gradient theorem shows that: $\frac{\partial \rho(\theta, s_0)}{\partial \theta} = \sum_s \mu_{\pi_\theta}(s \mid s_0) \sum_a \frac{\partial \pi_\theta(a \mid s)}{\partial \theta} Q_{\pi_\theta}(s, a)$, where $\mu_{\pi_\theta}(s \mid s_0) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0)$ is a discounted weighting of the states along the trajectories starting from s_0 . In practice, the policy gradient is estimated from samples along the on-policy stationary distribution. (Thomas 2014) showed that neglecting the discount factor in this stationary distribution makes the usual policy gradient estimator biased. However, correcting for this discrepancy also reduces data efficiency. For simplicity, we build on the framework of (Sutton et al. 2000) and discuss how to extend our results according to (Thomas 2014).

The options framework (Sutton, Precup, and Singh 1999; Precup 2000) formalizes the idea of temporally extended actions. A Markovian option $\omega \in \Omega$ is a triple $(\mathcal{I}_\omega, \pi_\omega, \beta_\omega)$ in which $\mathcal{I}_\omega \subseteq \mathcal{S}$ is an initiation set, π_ω is an *intra-option* policy, and $\beta_\omega : \mathcal{S} \rightarrow [0, 1]$ is a termination function. We also assume that $\forall s \in \mathcal{S}, \forall \omega \in \Omega : s \in \mathcal{I}_\omega$ (i.e., all options are available everywhere), an assumption made in the majority of option discovery algorithms. We will discuss how to dispense with this assumption in the final section. (Sutton, Precup, and Singh 1999; Precup 2000) show that an MDP endowed with a set of options becomes a Semi-Markov Decision Process (Puterman 1994, chapter 11), which has a corresponding optimal value function over options $V_\Omega(s)$ and option-value function $Q_\Omega(s, \omega)$. Learning and planning algorithms for MDPs have their counterparts in this setting. However, the existence of the underlying MDP offers the possibility of learning about many different options in parallel : this is the idea of *intra-option learning*, which we leverage in our work.

Learning Options

We adopt a continual perspective on the problem of learning options. At any time, we would like to distill all of the available experience into every component of our system: value function and policy over options, intra-option policies and termination functions. To achieve this goal, we focus on learning option policies and termination functions, assuming they are represented using differentiable parameterized function approximators.

We consider the *call-and-return* option execution model, in which an agent picks option ω according to its policy over options π_Ω , then follows the intra-option policy π_ω until termination (as dictated by β_ω), at which point this procedure is repeated. Let $\pi_{\omega, \theta}$ denote the intra-option policy of option ω parametrized by θ and $\beta_{\omega, \vartheta}$, the termination function of

ω parameterized by ϑ . We present two new results for learning options, obtained using as blueprint the policy gradient theorem (Sutton et al. 2000). Both results are derived under the assumption that the goal is to learn options that maximize the expected return in the current task. However, if one wanted to add extra information to the objective function, this could readily be done so long as it comes in the form of an additive differentiable function.

Suppose we aim to optimize directly the discounted return, expected over all the trajectories starting at a designated state s_0 and option ω_0 , then: $\rho(\Omega, \theta, \vartheta, s_0, \omega_0) = \mathbb{E}_{\Omega, \theta, \omega} [\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0, \omega_0]$. Note that this return depends on the policy over options, as well as the parameters of the option policies and termination functions. We will take gradients of this objective with respect to θ and ϑ . In order to do this, we will manipulate equations similar to those used in *intra-option* learning (Sutton, Precup, and Singh 1999, section 8). Specifically, the definition of the option-value function can be written as:

$$Q_\Omega(s, \omega) = \sum_a \pi_{\omega, \theta}(a \mid s) Q_U(s, \omega, a), \quad (1)$$

where $Q_U : \mathcal{S} \times \Omega \times \mathcal{A} \rightarrow \mathbb{R}$ is the value of executing an action in the context of a state-option pair:

$$Q_U(s, \omega, a) = r(s, a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s, a) U(\omega, s') \quad (2)$$

Note that the (s, ω) pairs lead to an augmented state space, cf. (Levy and Shimkin 2011). However, we will not work explicitly with this space; it is used only to simplify the derivation. The function $U : \Omega \times \mathcal{S} \rightarrow \mathbb{R}$ is called the option-value function *upon arrival*, (Sutton, Precup, and Singh 1999, equation 20). The value of executing ω upon entering a state s' is given by:

$$U(\omega, s') = (1 - \beta_{\omega, \vartheta}(s')) Q_\Omega(s', \omega) + \beta_{\omega, \vartheta}(s') V_\Omega(s') \quad (3)$$

Note that Q_U and U both depend on θ and ϑ , but we do not include these in the notation for clarity. The last ingredient required to derive policy gradients is the Markov chain along which the performance measure is estimated. The natural approach is to consider the chain defined in the augmented state space, because state-option pairs now play the role of regular states in a usual Markov chain. If option ω_t has been initiated or is executing at time t in state s_t , then the probability of transitioning to (s_{t+1}, ω_{t+1}) in one step is:

$$\mathbb{P}(s_{t+1}, \omega_{t+1} \mid s_t, \omega_t) = \sum_a \pi_{\omega_t, \theta}(a \mid s_t) \mathbb{P}(s_{t+1} \mid s_t, a) ((1 - \beta_{\omega_t, \vartheta}(s_{t+1})) \mathbf{1}_{\omega_t = \omega_{t+1}} + \beta_{\omega_t, \vartheta}(s_{t+1}) \pi_\Omega(\omega_{t+1} \mid s_{t+1})) \quad (4)$$

Clearly, the process given by (4) is homogeneous. Under mild conditions, and with options available everywhere, it is in fact ergodic, and a unique stationary distribution over state-option pairs exists.

We will now compute the gradient of the expected discounted return with respect to the parameters θ of the intra-

相相对于其自身的动作值函数是贪心的。

策略梯度方法 (Sutton等人2000; Konda和Tsitsiklis 2000) 通过执行随机梯度下降来优化给定参数化随机策略族 π_θ 上的性能目标, 从而解决寻找良好策略的问题。策略梯度定理 (Sutton等人2000) 提供了关于 θ 的平均奖励和折扣奖励目标的梯度表达式。在折扣设置中, 目标相对于指定的起始状态 (或分布) s_0 $\rho(s_0) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0]$ 定义: (θ, ϑ) 。策略梯度定理表明: $\frac{\partial \rho_{(\theta, \vartheta)}(s_0)}{\partial \theta} = \sum_s \mu_{\pi_\theta}(s \mid s_0) \sum_a \frac{\partial \pi_\theta(a \mid s)}{\partial \theta} Q_{\pi_\theta}(s, a)$, 其中 $\mu_{\pi_\theta}(s \mid s_0) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0)$ 是从 s_0 开始的轨迹上状态的折扣加权。在实践中, 策略梯度是从同策略平稳分布上的样本中估计的。(Thomas 2014) 表明, 在此平稳分布中忽略折扣因子会使通常的策略梯度估计器产生偏差。然而, 纠正这种差异也会降低数据效率。为简单起见, 我们基于 (Sutton等人2000) 的框架, 并讨论如何根据 (Thomas 2014) 扩展我们的结果。

选项框架 (Sutton、Precup和Singh 1999; Precup 2000) 形式化了时间扩展动作的概念。一个马尔可夫选项 $\omega \in \Omega$ 是一个三元组 $(\mathcal{I}_\omega, \pi_\omega, \beta_\omega)$, 其中 $\mathcal{I}_\omega \subseteq \mathcal{S}$ 是初始集, π_ω 是内部选项策略, 而 $\beta_\omega : \mathcal{S} \rightarrow [0, 1]$ 是终止函数。我们还假设 $\forall s \in \mathcal{S}, \forall \omega \in \Omega : s \in \mathcal{I}_\omega$ (即所有选项在所有地方都可用), 这是大多数选项发现算法所做的假设。我们将在最后一节讨论如何摒弃这一假设。(Sutton、Precup和Singh 1999; Precup 2000) 表明, 赋予一组选项的MDP变成了半马尔可夫决策过程 (Puterman 1994, 第11章), 该过程具有相应的选项上的最优价值函数 $V_\Omega(s)$ 和选项价值函数 $Q_\Omega(s, \omega)$ 。MDP的学习与规划算法在此设置中有其对应物。然而, 底层MDP的存在提供了并行学习许多不同选项的可能性: 这就是内部选项学习的思想, 我们在工作中利用了这一点。

学习选项

我们采用持续视角来解决学习选项的问题。在任何时刻, 我们都希望将所有可用的经验提炼到系统的每个组件中: 值函数和选项上的策略、选项内策略以及终止函数。为了实现这一目标, 我们专注于学习选项策略和终止函数, 并假设它们使用可微参数化函数逼近器来表示。

我们考虑调用-返回选项执行模型, 在该模型中, 智能体根据其选项上的策略 π_Ω 选择选项 ω , 然后遵循选项内策略 π_ω 直至终止 (由 β_ω 决定), 此时重复此过程。设 $\pi_{\omega, \theta}$ 表示由 θ 参数化的选项 ω 的选项内策略, 以及 $\beta_{\omega, \vartheta}$, 即终止函数

ω 由 ϑ 参数化。我们提出了两个关于学习选项的新结果, 这些结果以策略梯度定理 (Sutton等人2000) 为蓝图获得。这两个结果都是在假设目标是在当前任务中学习最大化期望回报的选项的前提下推导出来的。然而, 如果希望在目标函数中添加额外信息, 只要该信息以加性可微函数的形式出现, 就可以轻松实现。

假设我们旨在直接优化折扣回报, 该回报是在从指定状态 s_0 和选项 ω_0 开始的所有轨迹上期望得到的, 那么: $\rho(\Omega, \theta, \vartheta, s_0, \omega_0) = \mathbb{E}_{\Omega, \theta, \omega} [\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0, \omega_0]$ 。请注意, 该回报取决于选项上的策略, 以及选项策略和终止函数的参数。我们将针对 θ 和 ϑ 对该目标函数求梯度。为此, 我们将对与选项内学习 (Sutton、Precup和Singh 1999, 第8节) 中使用的方程进行类似的操作。具体来说, 选项值函数的定义可以写成:

$$Q_\Omega(s, \omega) = \sum_a \pi_{\omega, \theta}(a \mid s) Q_U(s, \omega, a), \quad (1)$$

其中 $Q_U : \mathcal{S} \times \Omega \times \mathcal{A} \rightarrow \mathbb{R}$ 是在状态-选项对情境下执行某个动作的价值:

$$Q_U(s, \omega, a) = r(s, a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s, a) U(\omega, s') \quad (2)$$

注意, (s, ω) 对构成了增广状态空间, 参见(Levy和Shimkin 2011)。然而, 我们不会显式地使用这个空间; 它仅用于简化推导过程。函数 $U : \Omega \times \mathcal{S} \rightarrow \mathbb{R}$ 被称为到达时的选项价值函数, (Sutton、Precup和Singh 1999, 公式20)。在进入状态 s' 时执行 ω 的价值由下式给出:

$$U(\omega, s') = (1 - \beta_{\omega, \vartheta}(s')) Q_\Omega(s', \omega) + \beta_{\omega, \vartheta}(s') V_\Omega(s') \quad (3)$$

注意, Q_U 和 U 都依赖于 θ 和 ϑ , 但为了简洁起见, 我们不在符号中体现这些依赖。推导策略梯度所需的最后一个要素是用于估计性能度量的马尔可夫链。自然的做法是考虑在增广状态空间中定义的链, 因为状态-选项对现在扮演了通常马尔可夫链中常规状态的角色。如果选项 ω_t 在时间 t 于状态 s_t 中已被启动或正在执行, 那么一步内转移到 (s_{t+1}, ω_{t+1}) 的概率为:

$$\mathbb{P}(s_{t+1}, \omega_{t+1} \mid s_t, \omega_t) = \sum_a \pi_{\omega_t, \theta}(a \mid s_t) \mathbb{P}(s_{t+1} \mid s_t, a) ((1 - \beta_{\omega_t, \vartheta}(s_{t+1})) \mathbf{1}_{\omega_t = \omega_{t+1}} + \beta_{\omega_t, \vartheta}(s_{t+1}) \pi_\Omega(\omega_{t+1} \mid s_{t+1})) \quad (4)$$

显然, 由 (4) 给出的过程是齐次的。在温和条件下, 且选项在所有地方都可用时, 它实际上是遍历的, 并且存在一个关于状态-选项对的唯一平稳分布。

我们现在将计算期望折扣回报相对于内部选项策略的参数 θ 的梯度, 假设它们是随机且可微的。由 (1, 2) 可知:

option policies, assuming that they are stochastic and differentiable. From (1, 2), it follows that:

$$\frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} = \left(\sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) \right) + \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \theta}.$$

We can further expand the right hand side using (3) and (4), which yields the following theorem:

Theorem 1 (Intra-Option Policy Gradient Theorem). *Given a set of Markov options with stochastic intra-option policies differentiable in their parameters θ , the gradient of the expected discounted return with respect to θ and initial condition (s_0, ω_0) is:*

$$\sum_{s, \omega} \mu_{\Omega}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) ,$$

where $\mu_{\Omega}(s, \omega | s_0, \omega_0)$ is a discounted weighting of state-option pairs along trajectories starting from (s_0, ω_0) : $\mu_{\Omega}(s, \omega | s_0, \omega_0) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s, \omega_t = \omega | s_0, \omega_0)$.

The proof is in the appendix. This gradient describes the effect of a local change at the primitive level on the global expected discounted return. In contrast, subgoal or pseudo-reward methods assume the objective of an option is simply to optimize its own reward function, ignoring how a proposed change would propagate in the overall objective.

We now turn our attention to computing gradients for the termination functions, assumed this time to be stochastic and differentiable in ϑ . From (1, 2, 3), we have:

$$\frac{\partial Q_{\Omega}(s, \omega)}{\partial \vartheta} = \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \vartheta}.$$

Hence, the key quantity is the gradient of U . This is a natural consequence of the call-and-return execution, in which the “goodness” of termination functions can only be evaluated upon entering the next state. The relevant gradient can be further expanded as:

$$\frac{\partial U(\omega, s')}{\partial \vartheta} = -\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) + \gamma \sum_{\omega'} \sum_{s''} P(s'', \omega' | s', \omega) \frac{\partial U(\omega', s'')}{\partial \vartheta} , \quad (5)$$

where A_{Ω} is the advantage function (Baird 1993) over options $A_{\Omega}(s', \omega) = Q_{\Omega}(s', \omega) - V_{\Omega}(s')$. Expanding $\frac{\partial U(\omega', s'')}{\partial \vartheta}$ recursively leads to a similar form as in theorem (1) but where the weighting of state-option pairs is now according to a Markov chain shifted by one time step: $\mu_{\Omega}(s_{t+1}, \omega_t | s_t, \omega_{t-1})$ (details are in the appendix).

Theorem 2 (Termination Gradient Theorem). *Given a set of Markov options with stochastic termination functions differentiable in their parameters ϑ , the gradient of the expected*

discounted return objective with respect to ϑ and the initial condition (s_1, ω_0) is:

$$-\sum_{s', \omega} \mu_{\Omega}(s', \omega | s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) ,$$

where $\mu_{\Omega}(s', \omega | s_1, \omega_0)$ is a discounted weighting of state-option pairs from (s_1, ω_0) : $\mu_{\Omega}(s, \omega | s_1, \omega_0) = \sum_{t=0}^{\infty} \gamma^t P(s_{t+1} = s, \omega_t = \omega | s_1, \omega_0)$.

The advantage function often appears in policy gradient methods (Sutton et al. 2000) when forming a *baseline* to reduce the variance in the gradient estimates. Its presence in that context has to do mostly with algorithm design. It is interesting that in our case, it follows as a direct consequence of the derivation and gives the theorem an intuitive interpretation: when the option choice is suboptimal with respect to the expected value over all options, the advantage function is negative and it drives the gradient corrections up, which increases the odds of terminating. After termination, the agent has the opportunity to pick a better option using π_{Ω} . A similar idea also underlies the *interrupting* execution model of options (Sutton, Precup, and Singh 1999) in which termination is forced whenever the value of $Q_{\Omega}(s', \omega)$ for the current option ω is less than $V_{\Omega}(s')$. (Mann, Mankowitz, and Mannor 2014) recently studied interrupting options through the lens of an *interrupting Bellman Operator* in a value-iteration setting. The termination gradient theorem can be interpreted as providing a gradient-based interrupting Bellman operator.

Algorithms and Architecture

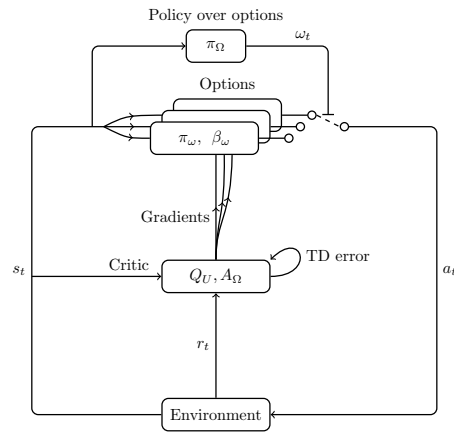


Figure 1: Diagram of the option-critic architecture. The option execution model is depicted by a *switch* $\perp$ over the *contacts* $\text{---}\circ$. A new option is selected according to π_{Ω} only when the current option terminates.

Based on theorems 1 and 2, we can now design a stochastic gradient descent algorithm for learning options. Using a two-timescale framework (Konda and Tsitsiklis 2000), we propose to learn the values at a *fast* timescale while updating the intra-option policies and termination functions at a *slower* rate.

选项策略是随机且可微的。由 (1, 2) 可知：

$$\frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} = \left(\sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) \right) + \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \theta}.$$

我们可以利用(3)和(4)进一步展开右侧，从而得到以下定理：

定理1（选项内策略梯度定理）。 给定一组具有随机选项内策略的马尔可夫选项， 这些策略在其参数 θ 上是可微的， 则期望折扣回报关于 θ 和初始条件 (s_0, ω_0) 的梯度为：

$$\sum_{s, \omega} \mu_{\Omega}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) ,$$

其中 $\mu_{\Omega}(s, \omega | s_0, \omega_0)$ 是从 (s_0, ω_0) 出发的轨迹上状态-选项对的折扣加权： $\mu_{\Omega}(s, \omega | s_0, \omega_0) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s, \omega_t = \omega | s_0, \omega_0)$ 。

证明见附录。该梯度描述了原始层级上的局部变化对全局期望折扣回报的影响。相比之下，子目标或伪奖励方法假设选项的目标仅仅是优化其自身的奖励函数，而忽略了提议的变化如何在整体目标中传播。

我们现在将注意力转向计算终止函数的梯度，这次假设它们在 ϑ 上是随机且可微的。由(1, 2, 3)式，我们有：

$$\frac{\partial Q_{\Omega}(s, \omega)}{\partial \vartheta} = \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \vartheta}.$$

因此，关键量是 U 的梯度。这是调用-返回执行的自然结果，在这种执行方式中，终止函数的“优劣”只能在进入下一个状态时才能被评估。相关的梯度可以进一步展开为：

$$\frac{\partial U(\omega, s')}{\partial \vartheta} = -\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) + \gamma \sum_{\omega'} \sum_{s''} P(s'', \omega' | s', \omega) \frac{\partial U(\omega', s'')}{\partial \vartheta} , \quad (5)$$

其中 A_{Ω} 是选项 $A_{\Omega}(s', \omega) = Q_{\Omega}(s', \omega) - V_{\Omega}(s')$ 上的优势函数 (Baird 1993)。递归地展开 $\frac{\partial U(\omega', s'')}{\partial \vartheta}$ 会得到与定理(1)类似的形式，但此时状态-选项对的加权是根据一个偏移了一个时间步的马尔可夫链进行的： $\mu_{\Omega}(s_{t+1}, \omega_t | s_t, \omega_{t-1})$ （详见附录）。**定理2**（终止梯度定理）。 给定一组马尔可夫选项， 其随机终止函数在其参数 ϑ 上可微， 则期望

折扣回报目标关于 ϑ 和初始条件 (s_1, ω_0) 的梯度为：

$$-\sum_{s', \omega} \mu_{\Omega}(s', \omega | s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) ,$$

其中 $\mu_{\Omega}(s', \omega | s_1, \omega_0)$ 是从 (s_1, ω_0) 出发的状态-选项对的折扣加权： $\mu_{\Omega}(s, \omega | s_1, \omega_0) = \sum_{t=0}^{\infty} \gamma^t P(s_{t+1} = s, \omega_t = \omega | s_1, \omega_0)$ 。

优势函数在策略梯度方法 (Sutton等人2000) 中经常出现，用于形成基线以减少梯度估计中的方差。它在那个语境中的存在主要与算法设计有关。有趣的是，在我们的案例中，它是推导的直接结果，并赋予定理一个直观的解释：当选项选择相对于所有选项的期望值而言是次优的，优势函数为负，这会推动梯度修正上升，从而增加终止的概率。终止后，智能体有机会使用 π_{Ω} 选择更好的选项。类似的想法也支撑了选项的中断执行模型 (Sutton、Precup和Singh 1999)，在该模型中，每当当前选项 ω 的 $Q_{\Omega}(s', \omega)$ 值小于 $V_{\Omega}(s')$ 时，就强制终止。(Mann、Mankowitz和Mannor 2014) 最近通过值迭代设置中的中断贝尔曼算子的视角研究了中断选项。终止梯度定理可以被解释为提供了一种基于梯度的中断贝尔曼算子。

算法与架构

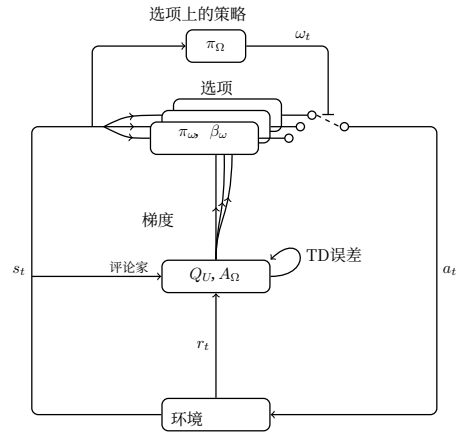


图1：选项-评论家架构示意图。选项执行模型通过一个开关 $\perp$ 作用于触点 $\text{---}\circ$ 来描绘。仅当当前选项终止时，才会根据 π_{Ω} 选择新选项。

基于定理1和2，我们现在可以设计一种用于学习选项的随机梯度下降算法。利用双时间尺度框架 (Konda和Tsitsiklis 2000)，我们建议在快速时间尺度上学习值，同时以较慢的速率更新选项内策略和终止函数。

We refer to the resulting system as an *option-critic architecture*, in reference to the actor-critic architectures (Sutton 1984). The intra-option policies, termination functions and policy over options belong to the *actor* part of the system while the *critic* consists of Q_U and A_Ω . The option-critic architecture does not prescribe how to obtain π_Ω since a variety of existing approaches would apply: using policy gradient methods at the SMDP level, with a planner over the options models, or using temporal difference updates. If π_Ω is the greedy policy over options, it follows from (2) that the corresponding one-step off-policy update target $g_t^{(1)}$ is:

$$g_t^{(1)} = r_{t+1} + \gamma \left((1 - \beta_{\omega_t, \vartheta}(s_{t+1})) \sum_a \pi_{\omega_t, \theta}(a | s_{t+1}) Q_U(s_{t+1}, \omega_t, a) + \beta_{\omega_t, \vartheta}(s_{t+1}) \max_{\omega} \sum_a \pi_{\omega, \theta}(a | s_{t+1}) Q_U(s_{t+1}, \omega, a) \right),$$

which is also the update target of the *intra-option Q-learning* algorithm of (Sutton, Precup, and Singh 1999). A prototypical implementation of option-critic which uses intra-option Q-learning is shown in Algorithm 1. The tabular setting is assumed only for clarity of presentation. We write α, α_θ and α_ϑ for the learning rates of the critic, intra-option policies and termination functions respectively.

Algorithm 1: Option-critic with tabular intra-option Q-learning

```

s ← s0
Choose ω according to an ε-soft policy over options
πΩ(s)
repeat
  Choose a according to πω, θ(a | s)
  Take action a in s, observe s', r

  1. Options evaluation:
  δ ← r − QU(s, ω, a)
  if s' is non-terminal then
    δ ← δ + γ(1 − βω, ϑ(s'))QΩ(s', ω) +
      γβω, ϑ(s') maxω̄ QΩ(s', ω̄)
  end
  QU(s, ω, a) ← QU(s, ω, a) + αδ

  2. Options improvement:
  θ ← θ + αθ  $\frac{\partial \log \pi_{\omega, \theta}(a | s)}{\partial \theta}$  QU(s, ω, a)
  ϑ ← ϑ − αϑ  $\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta}$  (QΩ(s', ω) − VΩ(s'))

  if βω, ϑ terminates in s' then
    choose new ω according to ε-soft(πΩ(s'))
    s ← s'
until s' is terminal

```

Learning Q_U in addition to Q_Ω is computationally wasteful both in terms of the number of parameters and samples. A practical solution is to only learn Q_Ω and derive an estimate of Q_U from it. Because Q_U is an expectation over next

states, $Q_U(s, \omega, a) = \mathbb{E}_{s' \sim P}[r(s, a) + \gamma U(\omega, s') | s, \omega, a]$, it follows that $g_t^{(1)}$ is an appropriate estimator. We chose this approach for our experiment with deep neural networks in the Arcade Learning Environment.

Experiments

We first consider a navigation task in the four-rooms domain (Sutton, Precup, and Singh 1999). Our goal is to evaluate the ability of a set of options learned fully autonomously to recover from a sudden change in the environment. (Sutton, Precup, and Singh 1999) presented a similar experiment for a set of pre-specified options; the options in our results have not been specified a priori.

Initially the goal is located in the east doorway and the initial state is drawn uniformly from all the other cells. After 1000 episodes, the goal moves to a random location in the lower right room. Primitive movements can fail with probability 1/3, in which case the agent transitions randomly to one of the empty adjacent cells. The discount factor was 0.99, and the reward was +1 at the goal and 0 otherwise. We chose to parametrize the intra-option policies with Boltzmann distributions and the terminations with sigmoid functions. The policy over options was learned using intra-option Q-learning. We also implemented primitive actor-critic (denoted AC-PG) using a Boltzmann policy. We also compared option-critic to a primitive SARSA agent using Boltzmann exploration and no eligibility traces. For all Boltzmann policies, we set the temperature parameter to 0.001. All the weights were initialized to zero.

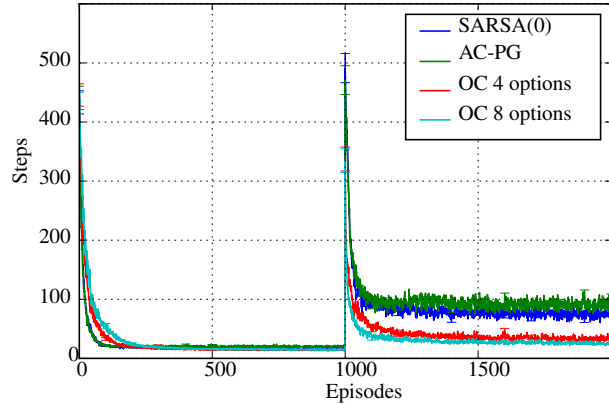


Figure 2: After a 1000 episodes, the goal location in the four-rooms domain is moved randomly. Option-critic (“OC”) recovers faster than the primitive actor-critic (“AC-PG”) and SARSA(0). Each line is averaged over 350 runs.

As can be seen in Figure 2, when the goal suddenly changes, the option-critic agent recovers faster. Furthermore, the initial set of options is learned *from scratch* at a rate comparable to primitive methods. Despite the simplicity of the domain, we are not aware of other methods which could have solved this task without incurring a cost much larger than when using primitive actions alone (McGovern and Barto 2001; Şimşek and Barto 2009).

我们将由此产生的系统称为选项-评论家架构，这是参照演员-评论家架构（萨顿1984）命名的。选项内策略、终止函数和选项上的策略属于系统的演员部分，而评论家则由 Q_U 和 A_Ω 组成。选项-评论家架构并未规定如何获取 π_Ω ，因为多种现有方法均可适用：在SMDP层级使用策略梯度方法，利用规划器处理选项模型，或使用时序差分更新。如果 π_Ω 是选项上的贪心策略，则由 (2) 可知，相应的单步离策略更新目标 $g_t^{(1)}$ 为：

$$g_t^{(1)} = r_{t+1} + \gamma \left((1 - \beta_{\omega_t, \vartheta}(s_{t+1})) \sum_a \pi_{\omega_t, \theta}(a | s_{t+1}) Q_U(s_{t+1}, \omega_t, a) + \beta_{\omega_t, \vartheta}(s_{t+1}) \max_{\omega} \sum_a \pi_{\omega, \theta}(a | s_{t+1}) Q_U(s_{t+1}, \omega, a) \right),$$

这也是 (Sutton, Precup和Singh 1999) 的选项内Q学习算法的更新目标。选项评论家的一个典型实现使用选项内Q学习，如算法1所示。假设表格设置仅为清晰展示。我们分别用 α, α_θ 和 α_ϑ 表示评论家、选项内策略和终止函数的学习率。

算法1：基于表格选项内Q学习的选项评论家

```

s ← s0
根据 ε-软性选项上的策略选择 ω
πΩ(s)
重复
  根据 πω, θ(a | s) 选择 a
  在 s 中采取行动 a, 观察 s', r

  1. 选项评估:
  δ ← r − QU(s, ω, a)
  如果 s' 是非终止状态 则
    δ ← δ + γ(1 − βω, ϑ(s'))QΩ(s', ω) +
      γβω, ϑ(s') maxω̄ QΩ(s', ω̄)
  end
  QU(s, ω, a) ← QU(s, ω, a) + αδ

  2. 选项改进:
  θ ← θ + αθ  $\frac{\partial \log \pi_{\omega, \theta}(a | s)}{\partial \theta}$  QU(s, ω, a)
  ϑ ← ϑ − αϑ  $\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta}$  (QΩ(s', ω) − VΩ(s'))

  如果 βω, ϑ 在 s' 处终止 则
    根据 ε-软性(πΩ(s')) 选择新的 ω
    s ← s'
直到 s' 为终止状态

```

除了 Q_Ω 之外，学习 Q_U 在参数数量和样本数量上都是计算上浪费的。一个实用的解决方案是只学习 Q_Ω ，并从中推导出 Q_U 的估计值。因为 Q_U 是对下一个

状态的期望， $Q_U(s, \omega, a) = \mathbb{E}_{s' \sim P}[r(s, a) + \gamma U(\omega, s') | s, \omega, a]$ ，因此 $g_t^{(1)}$ 是一个合适的估计器。我们在街机学习环境中使用深度神经网络的实验选择了这种方法。

实验

我们首先考虑四房间领域 (Sutton, Precup和Singh 1999) 中的导航任务。我们的目标是评估一组完全自主学习的选项在环境突然变化时恢复的能力。(Sutton, Precup和Singh 1999) 针对一组预先指定的选项提出了类似的实验；而我们的结果中的选项并非事先指定的。

最初，目标位于东侧门口，初始状态从所有其他单元格中均匀抽取。经过1000个回合后，目标移动到右下房间的随机位置。原始移动可能以概率 1/3失败，此时智能体随机转移到相邻的空单元格之一。折扣因子为0.99，奖励在目标处为 +1，其他情况为0。我们选择用玻尔兹曼分布参数化选项内策略，并用sigmoid函数参数化终止。选项上的策略通过选项内Q学习来学习。我们还实现了使用玻尔兹曼策略的原始演员-评论家（记为AC-PG）。我们还将选项评论家与使用玻尔兹曼探索且无资格迹的原始SARSA智能体进行了比较。对于所有玻尔兹曼策略，我们将温度参数设置为0.001。所有权重初始化为零。

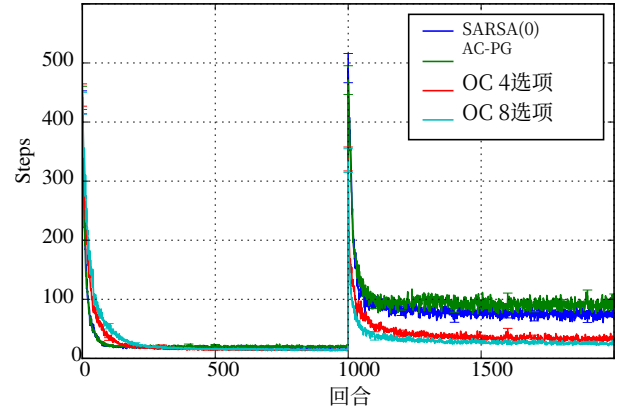


图2：经过1000个回合后，四房间领域中的目标位置被随机移动。选项评论家（“OC”）比原始演员-评论家（“AC-PG”）和SARSA(0)恢复得更快。每条线是350次运行的平均结果。

如图2所示，当目标突然改变时，选项-评论家智能体恢复得更快。此外，初始选项集是从零开始学习的，其速度与原始方法相当。尽管该领域简单，但我们不知道有其他方法能在不付出远大于仅使用原始动作的代价的情况下解决此任务 (McGovern和Barto 2001; Şimşek和Barto 2009)。

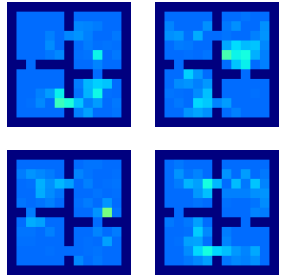


Figure 3: Termination probabilities for the option-critic agent learning with 4 options. The darkest color represents the *walls* in the environment while lighter colors encode higher termination probabilities.

In the two temporally extended settings, with 4 options and 8 options, termination events are more likely to occur near the doorways (Figure 3), agreeing with the intuition that they would be good subgoals. As opposed to (Sutton, Precup, and Singh 1999), we did not encode this knowledge ourselves but simply let the agents find options that would maximize the expected discounted return.

Pinball Domain

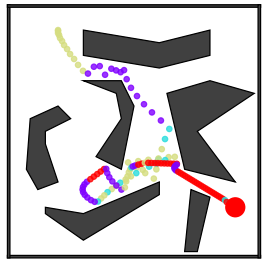


Figure 4: Pinball: Sample trajectory of the solution found after 250 episodes of training using 4 options. All options (color-coded) are used by the policy over options in successful trajectories. The initial state is in the top left corner and the goal is in the bottom right one (red circle).

In the Pinball domain (Konidaris and Barto 2009), a ball must be guided through a maze of arbitrarily shaped polygons to a designated target location. The state space is continuous over the position and velocity of the ball in the $x-y$ plane. At every step, the agent must choose among five discrete primitive actions: move the ball faster or slower, in the vertical or horizontal direction, or take the *null* action. Collisions with obstacles are elastic and can be used to the advantage of the agent. In this domain, a drag coefficient of 0.995 effectively stops ball movements after a finite number of steps when the null action is chosen repeatedly. Each thrust action incurs a penalty of -5 while taking no action costs -1 . The episode terminates with $+10000$ reward when the agent reaches the target. We interrupted any episode taking more than 10000 steps and set the discount factor to 0.99.

We used intra-option Q-learning in the critic with linear function approximation over Fourier bases (Konidaris et al.

2011) of order 3. We experimented with 2, 3 or 4 options. We used Boltzmann policies for the intra-option policies and linear-sigmoid functions for the termination functions. The learning rates were set to 0.01 for the critic and 0.001 for both the intra and termination gradients. We used an epsilon-greedy policy over options with $\epsilon = 0.01$.

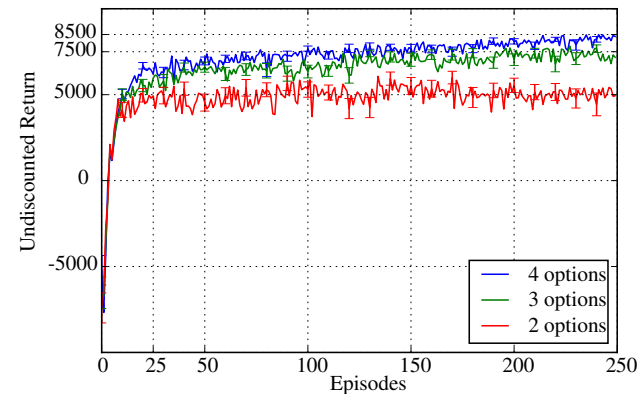


Figure 5: Learning curves in the Pinball domain.

In (Konidaris and Barto 2009), an option can only be used and updated after a *gestation* period of 10 episodes. As learning is fully integrated in option-critic, by 40 episodes a near optimal set of options had already been learned in all settings. From a qualitative point of view, the options exhibit temporal extension and specialization (fig. 4). We also observed that across many successful trajectories the *red* option would consistently be used in the vicinity of the goal.

Arcade Learning Environment

We applied the option-critic architecture in the Arcade Learning Environment (ALE) (Bellemare et al. 2013) using a deep neural network to approximate the critic and represent the intra-option policies and termination functions. We used the same configuration as (Mnih et al. 2013) for the first 3 convolutional layers of the network. We used 32 convolutional filters of size 8×8 and stride of 4 in the first layer, 64 filters of size 4×4 with a stride of 2 in the second and $64 \times 3 \times 3$ filters with a stride of 1 in the third layer. We then fed the output of the third layer into a dense shared layer of 512 neurons, as depicted in Figure 6. We fixed the learning rate for the intra-option policies and termination gradient to 0.00025 and used RMSProp for the critic.

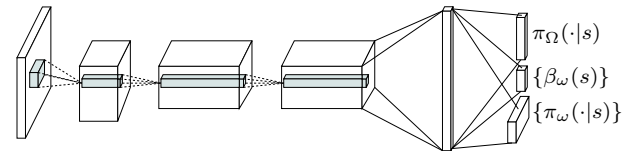


Figure 6: Deep neural network architecture. A concatenation of the last 4 images is fed through the convolutional layers, producing a dense representation shared across intra-option policies, termination functions and policy over options.

We represented the intra-option policies as linear-softmax

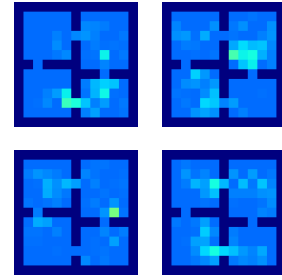


图3：使用4个选项学习的选项-评论家智能体的终止概率。最深的颜色代表环境中的墙壁，而较浅的颜色编码更高的终止概率。

在两种时间扩展设置中，即4个选项和8个选项的情况下，终止事件更可能发生在门口附近（图3），这与它们将是好的子目标的直觉一致。与（Sutton、Precup和Singh 1999）不同，我们没有自行编码这些知识，而是简单地让智能体寻找能够最大化期望折扣回报的选项。

弹球领域

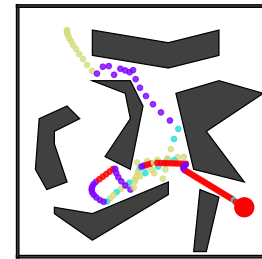


图4：弹球：使用4个选项训练250回合后找到的解决方案的示例轨迹。所有选项（按颜色编码）在成功轨迹中都被选项策略所使用。初始状态位于左上角，目标位于右下角（红色圆圈）。

在弹球领域（Konidaris和Barto 2009）中，一个球必须被引导穿过由任意形状多边形组成的迷宫，到达指定的目标位置。状态空间在 $x-y$ 平面上对球的位置和速度是连续的。在每一步中，智能体必须在五个离散的原始动作中选择：在垂直或水平方向上使球加速或减速，或者采取空动作。与障碍物的碰撞是弹性的，可以被智能体利用。在该领域中，阻力系数为0.995，当重复选择空动作时，球运动在有限步数后有效停止。每次推力动作产生 -5 的惩罚，而不采取任何动作则花费 -1 。当智能体到达目标时，回合以 $+10000$ 奖励终止。我们中断了任何超过 10000步的回合，并将折扣因子设置为0.99。

我们在评论家中使用了选项内Q学习，并采用基于傅里叶基的线性函数近似（Konidaris等人，

2011）的三阶形式。我们尝试了2、3或4个选项。对于选项内策略，我们使用了玻尔兹曼策略，对于终止函数，我们使用了线性S型函数。评论家的学习率设置为0.01，而选项内梯度和终止梯度的学习率均设置为0.001。我们使用了带有 $\epsilon = 0.01$ 的epsilon-贪心策略来选择选项。

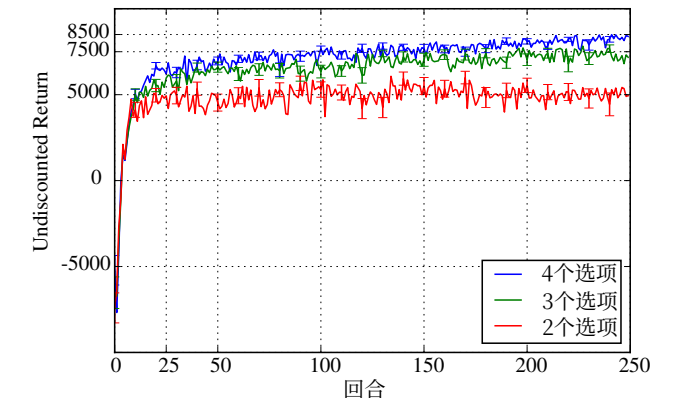


图5：弹球领域中的学习曲线。

在(Konidaris和Barto 2009)中，一个选项只能在10个回合的孕育期之后才能被使用和更新。由于学习完全集成在选项评论家中，到40个回合时，在所有设置中已经学习到了近似最优选项集。从定性的角度来看，这些选项表现出时间扩展和特化（图4）。我们还观察到，在许多成功的轨迹中，红色选项会一致地在目标附近被使用。

街机学习环境

我们在街机学习环境（ALE）中应用了选项-评论家架构（Bellemare 等人 2013），使用深度神经网络来近似评论家，并表示选项内策略和终止函数。我们采用了与（Mnih 等人 2013）相同的配置来处理网络的前3个卷积层。我们在第一层使用了 8×8 个大小为 8×8 的卷积滤波器，步长为4；第二层使用了64个大小为 4×4 的滤波器，步长为2；第三层使用了 $64 \times 3 \times 3$ 个滤波器，步长为1。然后，我们将第三层的输出馈送到一个包含512个神经元的密集共享层，如图6所示。我们将选项内策略和终止梯度的学习率固定为0.00025，并使用RMSProp来优化评论家。

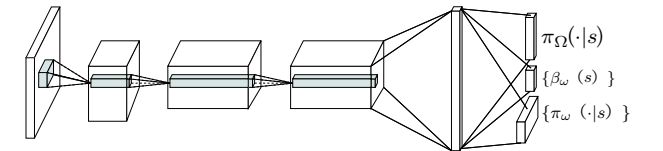


图6：深度神经网络架构。将最后4张图像的拼接结果输入卷积层，生成一个密集表示，该表示在选项内策略、终止函数和选项上的策略之间共享。

我们将选项内策略表示为线性softmax

of the fourth (dense) layer, so as to output a probability distribution over actions conditioned on the current observation. The termination functions were similarly defined using sigmoid functions, with one output neuron per termination.

The critic network was trained using intra-option Q-learning with experience replay. Option policies and terminations were updated on-line. We used an ϵ -greedy policy over options with $\epsilon = 0.05$ during the *test* phase (Mnih et al. 2013).

As a consequence of optimizing for the return, the termination gradient tends to *shrink* options over time. This is expected since in theory primitive actions are sufficient for solving any MDP. We tackled this issue by adding a small $\xi = 0.01$ term to the advantage function, used by the termination gradient: $A_{\Omega}(s, \omega) + \xi = Q_{\Omega}(s, \omega) - V_{\Omega}(s) + \xi$. This term has a regularization effect, by imposing an ξ -margin between the value estimate of an option and that of the “optimal” one reflected in V_{Ω} . This makes the advantage function positive if the value of an option is near the optimal one, thereby *stretching* it. A similar regularizer was proposed in (Mann, Mankowitz, and Mannor 2014).

As in (Mnih et al. 2016), we observed that the intra-option policies would quickly become deterministic. This problem seems to pertain to the use of policy gradient methods with deep neural networks in general, and not from option-critic itself. We applied the regularizer prescribed by (Mnih et al. 2016), by penalizing for low-entropy intra-option policies.

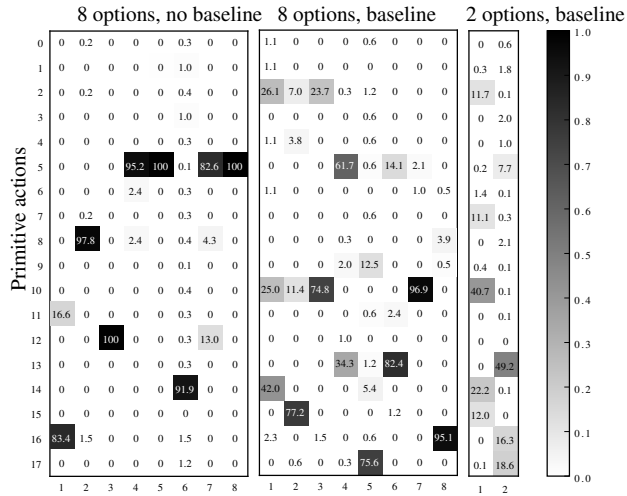


Figure 7: Seaquest: Using a baseline in the gradient estimators improves the distribution over actions in the intra-option policies, making them less deterministic. Each column represents one of the options learned in Seaquest. The vertical axis spans the 18 primitive actions of ALE. The empirical action frequencies are coded by intensity.

Finally, the baseline Q_{Ω} was added to the intra-option policy gradient estimator to reduce its variance. This change provided substantial improvements (Harb 2016) in the quality of the intra-option policy distributions and the overall agent performance as explained in Figure 7.

We evaluated option-critic in *Asterisk*, *Ms. Pacman*,

Seaquest and *Zaxxon*. For comparison, we allowed the system to learn for the same number of episodes as (Mnih et al. 2013) and fixed the parameters to the same values in all four domains. Despite having more parameters to learn, option-critic was capable of learning options that would achieve the goal in all games, from the ground up, within 200 episodes (Figure 8). In *Asterisk*, *Seaquest* and *Zaxxon*, option-critic surpassed the performance of the original DQN architecture based on primitive actions. The eight options learned in each game are learned fully end-to-end, in tandem with the feature representation, with no prior specification of a subgoal or pseudo-reward structure.

The solution found by option-critic was easy to interpret in the game of *Seaquest* when learning with only two options. We found that each option specialized in a behavior sequence which would include either the up or the down button. Figure 9 shows a typical transition from one option to the other, first going upward with option 0 then switching to option 1 downward. Options with a similar structure were also found in this game by (Krishnamurthy et al. 2016) using an option discovery algorithm based on graph partitioning.

Related Work

As option discovery has received a lot of attention recently, we now discuss in more detail the place of our approach with respect to others. (Comanici and Precup 2010) used a gradient-based approach for improving only the termination function of semi-Markov options; termination was modeled by a logistic distribution over a cumulative measure of the features observed since initiation. (Levy and Shimkin 2011) also built on policy gradient methods by constructing explicitly the augmented state space and treating stopping events as additional control actions. In contrast, we do not need to construct this (very large) space directly. (Silver and Ciosek 2012) dynamically chained options into longer temporal sequences by relying on compositionality properties. Earlier work on linear options (Sorg and Singh 2010) also used compositionality to plan using linear expectation models for options. Our approach also relies on the Bellman equations and compositionality, but in conjunction with policy gradient methods.

Several very recent papers also attempt to formulate option discovery as an optimization problem with solutions that are compatible with function approximation. (Daniel et al. 2016) learn return-optimizing options by treating the termination functions as hidden variables, and using EM to learn them. (Vezhnevets et al. 2016) consider the problem of learning options that have open-loop intra-option policies, also called *macro-actions*. As in classical planning, action sequences that are more frequent are cached. A mapping from states to action sequences is learned along with a *commitment module*, which triggers re-planning when necessary. In contrast, we use closed-loop policies throughout, which are reactive to state information and can provide better solutions. (Mankowitz, Mann, and Mannor 2016) propose a gradient-based option learning algorithm, assuming a particular structure for the initiation sets and termination functions. Under this framework, exactly one option is active in any partition of the state space. (Kulkarni et al. 2016)

第四（稠密）层的输出，以便在当前观测条件下输出动作上的概率分布。终止函数同样使用Sigmoid函数定义，每个终止对应一个输出神经元。

评论家网络使用内部选项Q学习与经验回放进行训练。选项策略和终止条件在线更新。在测试阶段，我们使用了 ϵ -贪心策略，其中 $\epsilon = 0.05$ （Mnih等人，2013）。

由于优化回报的结果，终止梯度倾向于随时间收缩选项。这是预期的，因为理论上原始动作足以解决任何马尔可夫决策过程。我们通过优势函数中添加一个小的 $\xi = 0.01$ 项来解决这个问题，该项用于终止梯度： $A_{\Omega}(s, \omega) + \xi = Q_{\Omega}(s, \omega) - V_{\Omega}(s) + \xi$ 。这一项具有正则化效应，通过在选项的价值估计与 V_{Ω} 中反映的“最优”选项的价值估计之间施加 ξ 间隔。这使得如果选项的价值接近最优值，优势函数为正，从而拉伸它。类似的正则化器在（Mann、Mankowitz和Mannor 2014）中提出。

如同（Mnih等人2016）所述，我们观察到选项内策略会迅速变得确定性。这个问题似乎普遍存在于使用深度神经网络进行策略梯度方法的过程中，而非选项评论家本身所致。我们采用了（Mnih等人2016）所建议的正则化器，通过对低熵选项内策略进行惩罚来实现。

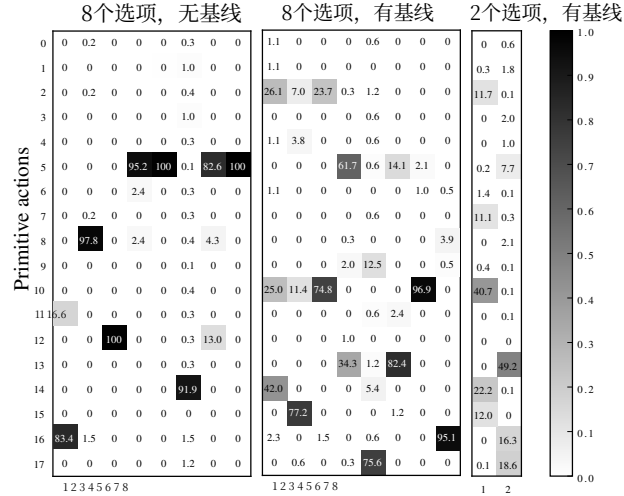


图7: Seaquest: 在梯度估计器中使用基线改善了选项内策略的动作分布，使其更少确定性。每列代表在Seaquest中学习到的一个选项。纵轴涵盖ALE的18个原始动作。经验动作频率通过强度编码。

最后，将基线 Q_{Ω} 添加到选项内策略梯度估计器中以降低其方差。这一改变在选项内策略分布质量和整体智能体性能方面带来了显著改进（Harb 2016），如图7所述。

我们在*Asterisk*、*Ms. Pacman*中评估了选项评论家，

Seaquest 和 *Zaxxon*。为了进行比较，我们允许系统学习与（Mnih 等人 2013）相同数量的回合，并在所有四个领域中固定参数为相同的值。尽管需要学习更多参数，选项评论家仍能够在200个回合内，从零开始在所有游戏中学习到能够实现目标的选项（图8）。在*Asterisk*、*Seaquest*和*Zaxxon*中，选项评论家的表现超越了基于原始动作的原始DQN架构。每个游戏中学到的八个选项完全端到端学习，与特征表示同步进行，无需事先指定子目标或伪奖励结构。

在仅学习两个选项时，选项评论家在*Seaquest*游戏中找到的解决方案易于解释。我们发现每个选项专门负责一个行为序列，该序列包含向上或向下按钮。图9展示了一个选项到另一个选项的典型转换，首先通过选项0向上移动，然后切换到选项1向下移动。Krishnamurthy等人（2016）也在此游戏中发现了类似结构的选项，他们使用了基于图划分的选项发现算法。

相关工作

由于选项发现近来受到了广泛关注，我们现在更详细地讨论我们的方法相对于其他方法的定位。(Comanici 和 Precup 2010) 使用了一种基于梯度的方法，仅改进半马尔可夫选项的终止函数；终止通过自启动以来观察到的特征的累积度量上的逻辑分布来建模。(Levy和Shimkin 2011) 也基于策略梯度方法，通过显式构建增广状态空间并将停止事件视为额外的控制动作。相比之下，我们不需要直接构建这个（非常大的）空间。(Silver 和 Ciosek 2012) 依靠组合性属性，将选项动态链接成更长的时序序列。早期关于线性选项的工作 (Sorg 和 Singh 2010) 也利用组合性，使用线性期望模型进行选项规划。我们的方法同样依赖于贝尔曼方程和组合性，但与策略梯度方法相结合。

最近的几篇论文也尝试将选项发现表述为一个优化问题，其解决方案与函数逼近兼容。Daniel等人，2016通过将终止函数视为隐变量，并使用EM算法来学习它们，从而学习回报优化选项。Vezhnevets等人，2016考虑了学习具有开环内部选项策略的选项问题，这些选项也被称为宏动作。与经典规划一样，更频繁的动作序列会被缓存。学习从状态到动作序列的映射，同时学习一个承诺模块，该模块在必要时触发重新规划。相比之下，我们全程使用闭环策略，这些策略对状态信息具有反应性，并能提供更好的解决方案。Mankowitz、Mann和Mannor，2016提出了一种基于梯度的选项学习算法，假设初始集和终止函数具有特定结构。在此框架下，状态空间的每个分区中恰好有一个选项处于活动状态。(Kulkarniet al. 2016)

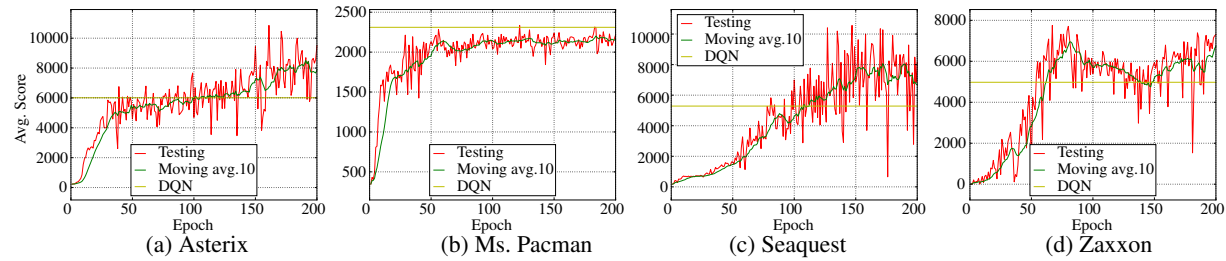


Figure 8: Learning curves in the Arcade Learning Environment. The same set of parameters was used across all four games: 8 options, 0.01终止正则化, 0.01熵正则化, 以及选项内策略梯度的基线。

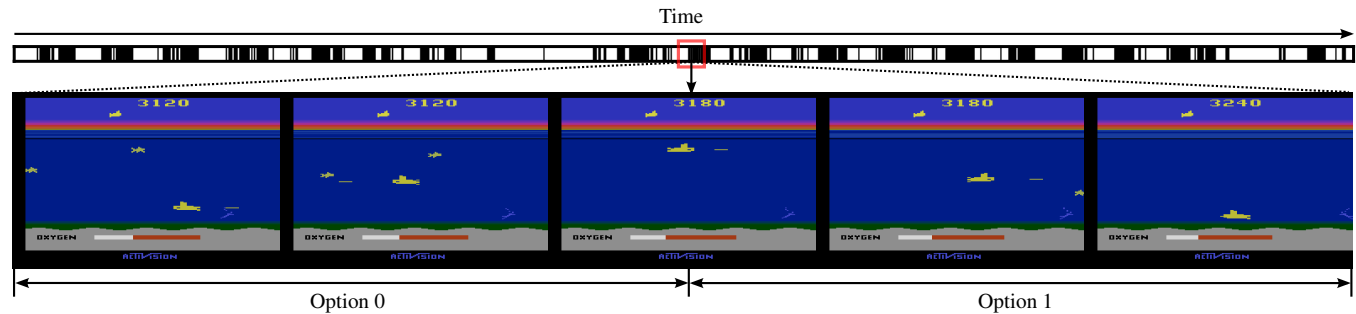


Figure 9: Up/down specialization in the solution found by option-critic when learning with 2 options in Seaquest. The top bar shows a trajectory in the game, with “white” representing a segment during which option 1 was active and “black” for option 2.

use the DQN framework to implement a gradient-based option learner, which uses intrinsic rewards to learn the internal policies of options, and extrinsic rewards to learn the policy over options. As opposed to our framework, descriptions of the subgoals are given as inputs to the option learners. Option-critic is conceptually general and does not require intrinsic motivation for learning the options.

Discussion

We developed a general gradient-based approach for learning simultaneously the intra-option policies and termination functions, as well as the policy over options, in order to optimize a performance objective for the task at hand. Our ALE experiments demonstrate successful end-to-end learning of options in the presence of nonlinear function approximation. As noted, our approach only requires specifying the number of options. However, if one wanted to use additional pseudo-rewards, the option-critic framework would easily accommodate it. In this case, the internal policies and termination function gradients would simply need to be taken with respect to the pseudo-rewards instead of the task reward. A simple instance of this idea, which we used in some of the experiments, is to use additional rewards to encourage options that are indeed temporally extended by adding a penalty whenever a switching event occurs. Our approach can work seamlessly with any other heuristic for biasing the set of options towards some desirable property (e.g. compositionality or sparsity), as long as it can be expressed as an additive reward structure. However, as seen in the results, such biasing is not necessary to produce good results.

The option-critic architecture relies on the policy gradient

theorem, and as discussed in (Thomas 2014), the gradient estimators can be biased in the discounted case. By introducing factors of the form $\gamma^t \prod_{i=1}^t (1 - \beta_i)$ in our updates (Thomas 2014, eq (3)), it would be possible to obtain unbiased estimates. However, we do not recommend this approach since the sample complexity of the unbiased estimators is generally too high and the biased estimators performed well in our experiments.

Perhaps the biggest remaining limitation of our work is the assumption that all options apply everywhere. In the case of function approximation, a natural extension to initiation sets is to use a classifier over features, or some other form of function approximation. As a result, determining which options are allowed may have similar cost to evaluating a policy over options (unlike in the tabular setting, where options with sparse initiation sets lead to faster decisions). This is akin to eligibility traces, which are more expensive than using no trace in the tabular case, but have the same complexity with function approximation. If initiation sets are to be learned, the main constraint that needs to be added is that the options and the policy over them lead to an ergodic chain in the augmented state-option space. This can be expressed as a flow condition that links initiation sets with terminations. The precise description of this condition, as well as sparsity regularization for initiation sets, is left for future work.

Acknowledgements

The authors gratefully acknowledge financial support for this work by the National Science and Engineering Research Council of Canada (NSERC) and the Fonds de recherche du Quebec - Nature et Technologies (FRQNT).

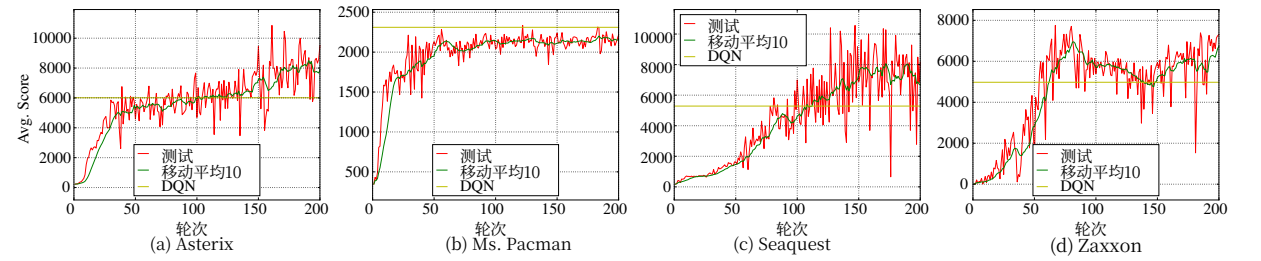


图8: 街机学习环境中的学习曲线。所有四个游戏均使用相同的参数集: 8个选项, 0.01终止正则化, 0.01熵正则化, 以及选项内策略梯度的基线。

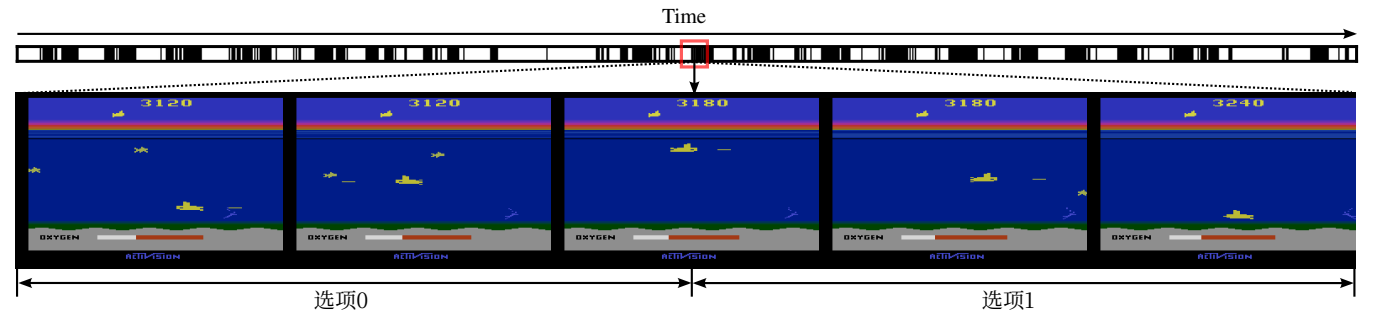


图9: 在Seaquest中使用2个选项进行选项-评论家学习时发现的上下特化。顶部条带显示了游戏中的一条轨迹, 其中“白色”表示选项1处于活动状态的片段, “黑色”表示选项2处于活动状态的片段。

使用DQN框架实现基于梯度的选项学习器, 该学习器利用内在奖励来学习选项的内部策略, 并利用外在奖励来学习选项之上的策略。与我们的框架不同, 子目标的描述作为输入提供给选项学习器。选项评论家在概念上是通用的, 不需要内在动机来学习选项。

讨论

我们开发了一种通用的基于梯度的方法, 用于同时学习选项内策略和终止函数, 以及选项上的策略, 以优化当前任务的性能目标。我们的ALE实验证明了在非线性函数逼近存在的情况下, 选项的端到端学习是成功的。如前所述, 我们的方法仅需要指定选项的数量。然而, 如果希望使用额外的伪奖励, 选项-评论家框架可以轻松地适应这一需求。在这种情况下, 内部策略和终止函数的梯度只需相对于伪奖励而非任务奖励来计算。我们在部分实验中使用的这一思想的一个简单实例是, 通过添加额外的奖励来鼓励确实在时间上扩展的选项, 即在每次发生切换事件时添加惩罚。我们的方法可以与任何其他启发式方法无缝配合, 只要这些方法能将选项集合偏向某些期望属性 (例如组合性或稀疏性), 并且可以表示为加性奖励结构。然而, 从结果中可以看出, 这种偏置并非产生良好结果的必要条件。

选项-评论家架构依赖于策略梯度

定理, 并且如 (Thomas 2014) 中所讨论的, 在折扣情形下梯度估计器可能存在偏差。通过在更新中引入形如 $\gamma^t \prod_{i=1}^t (1 - \beta_i)$ 的因子 (Thomas 2014, 公式(3)), 可以获得无偏估计。然而, 我们不推荐这种方法, 因为无偏估计器的样本复杂度通常过高, 而有偏估计器在我们的实验中表现良好。

也许我们工作中最大的剩余局限是假设所有选项适用于所有地方。在函数逼近的情况下, 对初始集的一个自然扩展是使用特征分类器, 或其他形式的函数逼近。因此, 确定哪些选项被允许可能具有与评估选项策略相似的成本 (不像在表格设置中, 具有稀疏初始集的选项能带来更快的决策)。这类似于资格迹, 在表格情况下比不使用迹更昂贵, 但在函数逼近下具有相同的复杂度。如果要学习初始集, 需要添加的主要约束是选项及其上的策略在增广状态-选项空间中导致遍历链。这可以表示为将初始集与终止联系起来的流条件。该条件的精确描述, 以及初始集的稀疏正则化, 留待未来工作。

致谢

作者衷心感谢加拿大自然科学与工程研究委员会 (NSERC) 和魁北克自然与技术研究基金 (FRQNT) 对本工作的财务支持。

Appendix

Augmented Process

If ω_t has been initiated or is executing at time t , then the discounted probability of transitioning to (s_{t+1}, ω_{t+1}) is:

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_{t+1} | s_t, \omega_t) = \sum_a \pi_{\omega_t}(a | s_t) \gamma P(s_{t+1} | s_t, a) ((1 - \beta_{\omega_t}(s_{t+1})) \mathbf{1}_{\omega_t = \omega_{t+1}} + \beta_{\omega_t}(s_{t+1}) \pi_{\Omega}(\omega_{t+1} | s_{t+1})) .$$

When conditioning the process from (s_t, ω_{t-1}) , the discounted probability of transitioning to s_{t+1}, ω_t is:

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_t | s_t, \omega_{t-1}) = ((1 - \beta_{\omega_{t-1}}(s_t)) \mathbf{1}_{\omega_t = \omega_{t-1}} + \beta_{\omega_{t-1}}(s_t) \pi_{\Omega}(\omega_t | s_t)) \sum_a \pi_{\omega_t}(a | s_t) \gamma P(s_{t+1} | s_t, a) .$$

More generally, the k -steps discounted probabilities can be expressed recursively as follows:

$$P_{\gamma}^{(k)}(s_{t+k}, \omega_{t+k} | s_t, \omega_t) = \sum_{s_{t+1}} \sum_{\omega_{t+1}} ($$

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_{t+1} | s_t, \omega_t) P_{\gamma}^{(k-1)}(s_{t+k}, \omega_{t+k} | s_{t+1}, \omega_{t+1}) ,$$

$$P_{\gamma}^{(k)}(s_{t+k}, \omega_{t+k-1} | s_t, \omega_{t-1}) = \sum_{s_{t+1}} \sum_{\omega_t} ($$

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_t | s_t, \omega_{t-1}) P_{\gamma}^{(k-1)}(s_{t+k}, \omega_{t+k-1} | s_{t+1}, \omega_t) .$$

Proof of the Intra-Option Policy Gradient Theorem

Taking the gradient of the option-value function:

$$\begin{aligned} \frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} &= \frac{\partial}{\partial \theta} \sum_a \pi_{\omega, \theta}(a | s) Q_U(s, \omega, a) \\ &= \sum_a \left(\frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \pi_{\omega, \theta}(a | s) \frac{\partial Q_U(s, \omega, a)}{\partial \theta} \right) \\ &= \sum_a \left(\frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \theta} \right) , \quad (6) \\ \frac{\partial U(\omega, s')}{\partial \theta} &= (1 - \beta_{\omega, \vartheta}(s')) \frac{\partial Q_{\Omega}(s', \omega)}{\partial \theta} + \beta_{\omega, \vartheta}(s') \frac{\partial V_{\Omega}(s')}{\partial \theta} \\ &= (1 - \beta_{\omega, \vartheta}(s')) \frac{\partial Q_{\Omega}(s', \omega)}{\partial \theta} + \beta_{\omega, \vartheta}(s') \sum_{\omega'} \pi_{\Omega}(\omega' | s') \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_{\omega'} ((1 - \beta_{\omega, \vartheta}(s')) \mathbf{1}_{\omega' = \omega} + \beta_{\omega, \vartheta}(s') \pi_{\Omega}(\omega' | s')) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} . \quad (7) \end{aligned}$$

where (7) follows from the assumption that θ only appears in the intra-option policies. Substituting (7) into (6) yields a recursion which, using the previous remarks about augmented process can be transformed into:

$$\begin{aligned} \frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} &= \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \sum_{\omega'} \left(\beta_{\omega, \vartheta}(s') \pi_{\Omega}(\omega' | s') + (1 - \beta_{\omega, \vartheta}(s')) \mathbf{1}_{\omega' = \omega} \right) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \sum_{s'} \sum_{\omega'} P_{\gamma}^{(1)}(s', \omega' | s, \omega) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_{k=0}^{\infty} \sum_{s', \omega'} P_{\gamma}^{(k)}(s', \omega' | s, \omega) \sum_a \frac{\partial \pi_{\omega', \theta}(a | s')}{\partial \theta} Q_U(s', \omega', a) . \end{aligned}$$

The gradient of the expected discounted return with respect to θ is then:

$$\frac{\partial Q_{\Omega}(s_0, \omega_0)}{\partial \theta} = \sum_{s, \omega} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) = \sum_{s, \omega} \mu_{\Omega}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) .$$

Proof of the Termination Gradient Theorem

The expected sum of discounted rewards starting from (s_1, ω_0) is given by:

$$U(\omega_0, s_1) = \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \middle| s_1, \omega_0 \right] .$$

We start by expanding U as follows:

$$\begin{aligned} U(\omega, s') &= (1 - \beta_{\omega, \vartheta}(s')) Q_{\Omega}(s', \omega) + \beta_{\omega, \vartheta}(s') V_{\Omega}(s') \\ &= (1 - \beta_{\omega, \vartheta}(s')) \sum_a \pi_{\omega, \theta}(a | s') \left(r(s', a) + \sum_{s''} \gamma P(s'' | s', a) U(\omega, s'') \right) \\ &\quad + \beta_{\omega, \vartheta}(s') \sum_{\omega'} \pi_{\Omega}(\omega' | s') \sum_a \pi_{\omega', \theta}(a | s') \left(r(s', a) + \sum_{s''} \gamma P(s'' | s', a) U(\omega', s'') \right) . \end{aligned}$$

The gradient of U is then:

$$\begin{aligned} \frac{\partial U(\omega, s')}{\partial \vartheta} &= \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} \underbrace{(V_{\Omega}(s') - Q_{\Omega}(s', \omega))}_{-A_{\Omega}(s', \omega)} + (1 - \beta_{\omega, \vartheta}(s')) \sum_a \pi_{\omega, \theta}(a | s') \sum_{s''} \gamma P(s'' | s', a) \frac{\partial U(\omega, s'')}{\partial \vartheta} . \end{aligned}$$

附录

增广过程

如果 ω_t 已在时间 t 启动或正在执行，则转换到 (s_{t+1}, ω_{t+1}) 的折扣概率为：

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_{t+1} | s_t, \omega_t) = \sum_a \pi_{\omega_t}(a | s_t) \gamma P(s_{t+1} | s_t, a) ((1 - \beta_{\omega_t}(s_{t+1})) \mathbf{1}_{\omega_t = \omega_{t+1}} + \beta_{\omega_t}(s_{t+1}) \pi_{\Omega}(\omega_{t+1} | s_{t+1})) .$$

当从 (s_t, ω_{t-1}) 对该过程进行条件化时， 转移到 s_{t+1}, ω_t 的折扣概率为：

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_t | s_t, \omega_{t-1}) = ((1 - \beta_{\omega_{t-1}}(s_t)) \mathbf{1}_{\omega_t = \omega_{t-1}} + \beta_{\omega_{t-1}}(s_t) \pi_{\Omega}(\omega_t | s_t)) \sum_a \pi_{\omega_t}(a | s_t) \gamma P(s_{t+1} | s_t, a) .$$

更一般地， k 步折扣概率可以递归地表示如下：

$$P_{\gamma}^{(k)}(s_{t+k}, \omega_{t+k} | s_t, \omega_t) = \sum_{s_{t+1}} \sum_{\omega_{t+1}} ($$

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_{t+1} | s_t, \omega_t) P_{\gamma}^{(k-1)}(s_{t+k}, \omega_{t+k} | s_{t+1}, \omega_{t+1}) ,$$

$$P_{\gamma}^{(k)}(s_{t+k}, \omega_{t+k-1} | s_t, \omega_{t-1}) = \sum_{s_{t+1}} \sum_{\omega_t} ($$

$$P_{\gamma}^{(1)}(s_{t+1}, \omega_t | s_t, \omega_{t-1}) P_{\gamma}^{(k-1)}(s_{t+k}, \omega_{t+k-1} | s_{t+1}, \omega_t) .$$

选项内策略梯度定理的证明对选项价值函数求梯度：

$$\begin{aligned} \frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} &= \frac{\partial}{\partial \theta} \sum_a \pi_{\omega, \theta}(a | s) Q_U(s, \omega, a) \\ &= \sum_a \left(\frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \pi_{\omega, \theta}(a | s) \frac{\partial Q_U(s, \omega, a)}{\partial \theta} \right) \\ &= \sum_a \left(\frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \frac{\partial U(\omega, s')}{\partial \theta} \right) , \quad (6) \\ \frac{\partial U(\omega, s')}{\partial \theta} &= (1 - \beta_{\omega, \vartheta}(s')) \frac{\partial Q_{\Omega}(s', \omega)}{\partial \theta} + \beta_{\omega, \vartheta}(s') \frac{\partial V_{\Omega}(s')}{\partial \theta} \\ &= (1 - \beta_{\omega, \vartheta}(s')) \frac{\partial Q_{\Omega}(s', \omega)}{\partial \theta} + \beta_{\omega, \vartheta}(s') \sum_{\omega'} \pi_{\Omega}(\omega' | s') \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_{\omega'} ((1 - \beta_{\omega, \vartheta}(s')) \mathbf{1}_{\omega' = \omega} + \beta_{\omega, \vartheta}(s') \pi_{\Omega}(\omega' | s')) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} . \quad (7) \end{aligned}$$

其中（7）由假设 θ 仅出现在选项内策略中得出。将

（7）代入（6）得到一个递归式，利用前述关于增广过程的说明，可将其转化为：

$$\begin{aligned} \frac{\partial Q_{\Omega}(s, \omega)}{\partial \theta} &= \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \sum_a \pi_{\omega, \theta}(a | s) \sum_{s'} \gamma P(s' | s, a) \sum_{\omega'} \left(\beta_{\omega, \vartheta}(s') \pi_{\Omega}(\omega' | s') + (1 - \beta_{\omega, \vartheta}(s')) \mathbf{1}_{\omega' = \omega} \right) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) + \sum_{s'} \sum_{\omega'} P_{\gamma}^{(1)}(s', \omega' | s, \omega) \frac{\partial Q_{\Omega}(s', \omega')}{\partial \theta} \\ &= \sum_{k=0}^{\infty} \sum_{s', \omega'} P_{\gamma}^{(k)}(s', \omega' | s, \omega) \sum_a \frac{\partial \pi_{\omega', \theta}(a | s')}{\partial \theta} Q_U(s', \omega', a) . \end{aligned}$$

期望折扣回报关于 θ 的梯度为：

$$\begin{aligned} \frac{\partial Q_{\Omega}(s_0, \omega_0)}{\partial \theta} &= \sum_{s, \omega} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) \\ &= \sum_{s, \omega} \mu_{\Omega}(s, \omega | s_0, \omega_0) \sum_a \frac{\partial \pi_{\omega, \theta}(a | s)}{\partial \theta} Q_U(s, \omega, a) . \end{aligned}$$

终止梯度定理的证明

从 (s_1, ω_0) 开始的期望折扣奖励总和由下式给出：

$$U(\omega_0, s_1) = \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \middle| s_1, \omega_0 \right] .$$

我们首先将 U 展开如下：

$$\begin{aligned} U(\omega, s') &= (1 - \beta_{\omega, \vartheta}(s')) Q_{\Omega}(s', \omega) + \beta_{\omega, \vartheta}(s') V_{\Omega}(s') \\ &= (1 - \beta_{\omega, \vartheta}(s')) \sum_a \pi_{\omega, \theta}(a | s') \left(r(s', a) + \sum_{s''} \gamma P(s'' | s', a) U(\omega, s'') \right) \\ &\quad + \beta_{\omega, \vartheta}(s') \sum_{\omega'} \pi_{\Omega}(\omega' | s') \sum_a \pi_{\omega', \theta}(a | s') \left(r(s', a) + \sum_{s''} \gamma P(s'' | s', a) U(\omega', s'') \right) . \end{aligned}$$

U 的梯度为：

$$\begin{aligned} \frac{\partial U(\omega, s')}{\partial \vartheta} &= \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} \underbrace{(V_{\Omega}(s') - Q_{\Omega}(s', \omega))}_{-A_{\Omega}(s', \omega)} + (1 - \beta_{\omega, \vartheta}(s')) \sum_a \pi_{\omega, \theta}(a | s') \sum_{s''} \gamma P(s'' | s', a) \frac{\partial U(\omega, s'')}{\partial \vartheta} . \end{aligned}$$

Using the structure of the augmented process:

$$\begin{aligned} \frac{\partial U(\omega, s')}{\partial \vartheta} &= -\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) + \\ &\quad \sum_{\omega'} \sum_{s''} P_{\gamma}^{(1)}(s'', \omega' \mid s', \omega) \frac{\partial U(\omega', s'')}{\partial \vartheta} \\ &= -\sum_{\omega', s''} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s'', \omega' \mid s', \omega) \frac{\partial \beta_{\omega', \vartheta}(s'')}{\partial \vartheta} A_{\Omega}(s'', \omega') \quad . \end{aligned}$$

We finally obtain:

$$\begin{aligned} \frac{\partial U(\omega_0, s_1)}{\partial \vartheta} &= \\ &- \sum_{\omega', s'} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s', \omega \mid s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) \\ &= -\sum_{\omega, s'} \mu_{\Omega}(s', \omega \mid s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) \quad . \end{aligned}$$

References

- Baird, L. C. 1993. Advantage updating. Technical Report WL–TR-93-1146, Wright Laboratory.
- Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research* 47:253–279.
- Comanici, G., and Precup, D. 2010. Optimal policy switching algorithms for reinforcement learning. In *AAMAS*, 709–714.
- Şimşek, O., and Barto, A. G. 2009. Skill characterization based on betweenness. In *NIPS 21*, 1497–1504.
- Daniel, C.; van Hoof, H.; Peters, J.; and Neumann, G. 2016. Probabilistic inference for determining options in reinforcement learning. *Machine Learning, Special Issue* 104(2):337–357.
- Harb, J. 2016. Learning options in deep reinforcement learning. Master’s thesis, McGill University.
- Konda, V. R., and Tsitsiklis, J. N. 2000. Actor-critic algorithms. In *NIPS 12*, 1008–1014.
- Konidaris, G., and Barto, A. 2009. Skill discovery in continuous reinforcement learning domains using skill chaining. In *NIPS 22*, 1015–1023.
- Konidaris, G.; Kuindersma, S.; Grupen, R. A.; and Barto, A. G. 2011. Autonomous skill acquisition on a mobile manipulator. In *AAAI*.
- Krishnamurthy, R.; Lakshminarayanan, A. S.; Kumar, P.; and Ravindran, B. 2016. Hierarchical reinforcement learning using spatio-temporal abstractions and deep neural networks. *CoRR* abs/1605.05359.
- Kulkarni, T.; Narasimhan, K.; Saeedi, A.; and Tenenbaum, J. 2016. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *NIPS 29*.
- Levy, K. Y., and Shimkin, N. 2011. Unified inter and intra options learning using policy gradient methods. In *EWRL*, 153–164.

- Mankowitz, D. J.; Mann, T. A.; and Mannor, S. 2016. Adaptive skills, adaptive partitions (ASAP). In *NIPS 29*.
- Mann, T. A.; Mankowitz, D. J.; and Mannor, S. 2014. Time-regularized interrupting options (TRIO). In *ICML*, 1350–1358.
- Mann, T. A.; Mannor, S.; and Precup, D. 2015. Approximate value iteration with temporally extended actions. *Journal of Artificial Intelligence Research* 53:375–438.
- McGovern, A., and Barto, A. G. 2001. Automatic discovery of subgoals in reinforcement learning using diverse density. In *ICML*, 361–368.
- Menache, I.; Mannor, S.; and Shimkin, N. 2002. Q-cut - dynamic discovery of sub-goals in reinforcement learning. In *ECML*, 295–306.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. A. 2013. Playing atari with deep reinforcement learning. *CoRR* abs/1312.5602.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T. P.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *ICML*.
- Niekum, S. 2013. *Semantically Grounded Learning from Unstructured Demonstrations*. Ph.D. Dissertation, University of Massachusetts, Amherst.
- Precup, D. 2000. *Temporal abstraction in reinforcement learning*. Ph.D. Dissertation, University of Massachusetts, Amherst.
- Puterman, M. L. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc.
- Silver, D., and Ciosek, K. 2012. Compositional planning using optimal option models. In *ICML*.
- Sorg, J., and Singh, S. P. 2010. Linear options. In *AAMAS*, 31–38.
- Stolle, M., and Precup, D. 2002. Learning options in reinforcement learning. In *Abstraction, Reformulation and Approximation, 5th International Symposium, SARA Proceedings*, 212–223.
- Sutton, R. S.; McAllester, D. A.; Singh, S. P.; and Mansour, Y. 2000. Policy gradient methods for reinforcement learning with function approximation. In *NIPS 12*. 1057–1063.
- Sutton, R. S.; Precup, D.; and Singh, S. P. 1999. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112(1-2):181–211.
- Sutton, R. S. 1984. *Temporal Credit Assignment in Reinforcement Learning*. Ph.D. Dissertation.
- Thomas, P. 2014. Bias in natural actor-critic algorithms. In *ICML*, 441–448.
- Vezhnevets, A. S.; Mnih, V.; Agapiou, J.; Osindero, S.; Graves, A.; Vinyals, O.; and Kavukcuoglu, K. 2016. Strategic attentive writer for learning macro-actions. In *NIPS 29*.

利用增广过程的结构：

$$\begin{aligned} \frac{\partial U(\omega, s')}{\partial \vartheta} &= -\frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) + \\ &\quad \sum_{\omega'} \sum_{s''} P_{\gamma}^{(1)}(s'', \omega' \mid s', \omega) \frac{\partial U(\omega', s'')}{\partial \vartheta} \\ &= -\sum_{\omega', s''} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s'', \omega' \mid s', \omega) \frac{\partial \beta_{\omega', \vartheta}(s'')}{\partial \vartheta} A_{\Omega}(s'', \omega') \quad . \end{aligned}$$

我们最终得到：

$$\begin{aligned} \frac{\partial U(\omega_0, s_1)}{\partial \vartheta} &= \\ &- \sum_{\omega, s'} \sum_{k=0}^{\infty} P_{\gamma}^{(k)}(s', \omega \mid s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) \\ &= -\sum_{\omega, s'} \mu_{\Omega}(s', \omega \mid s_1, \omega_0) \frac{\partial \beta_{\omega, \vartheta}(s')}{\partial \vartheta} A_{\Omega}(s', \omega) \quad . \end{aligned}$$

参考文献

- Baird, L. C. 1993. 优势更新. 技术报告 WL–TR-93-1146, 赖特实验室.
- Bellemare, M. G.; Naddaf, Y.; Veness, J.; 和 Bowling, M. 2013. 街机学习环境：通用智能体的评估平台。 人工智能研究杂志47:253–279.
- Comanici, G., 和 Precup, D. 2010. 强化学习中的最优策略切换算法。在 *AAMAS*, 709–714.
- Sims.ek, O., 和 Barto, A. G. 2009. 基于介数中心性的技能表征。在 *NIPS 21*, 1497–1504.
- Daniel, C.; van Hoof, H.; Peters, J.; 和 Neumann, G. 2016. 用于确定强化学习选项的概率推断。 机器学习, 特刊104(2):337–357.
- Harb, J. 2016. 深度强化学习中的学习选项。 硕士论文, 麦吉尔大学.
- Konda, V. R., 和 Tsitsiklis, J. N. 2000. 演员-评论家算法。在 *NIPS 12*, 1008–1014.
- Konidaris, G., 和 Barto, A. 2009. 使用技能链在连续强化学习领域中发现技能。在 *NIPS 22*, 1015–1023.
- Konidaris, G.; Kuindersma, S.; Grupen, R. A.; 和 Barto, A. G. 2011. 移动机械臂上的自主技能获取。在 *AAAI*.
- Krishnamurthy, R.; Lakshminarayanan, A. S.; Kumar, P.; 和 Ravindran, B. 2016. 使用时空抽象和深度神经网络的分层强化学习。 *CoRR*abs/1605.05359.
- Kulkarni, T.; Narasi mhan, K.; Saeedi, A.; 和 Tenenbaum, J. 2016. 分层深度强化学习：整合时间抽象和内在动机。在 *NIPS 29*.
- Levy, K. Y., 和 Shimkin, N. 2011. 使用策略梯度方法的统一选项间和选项内学习。在 *EWRL*, 153–164.

- Mankowitz, D. J.; Mann, T. A.; 和 Mannor, S. 2016. 自适应技能, 自适应分区 (ASAP)。在 *NIPS 29*.
- Mann, T. A.; Mankowitz, D. J.; 和 Mannor, S. 2014. 时间正则化中断选项 (TRIO)。在 *ICML*, 1350–1358.
- Mann, T. A.; Mannor, S.; 和 Precup, D. 2015. 使用时间扩展动作的近似值迭代。 人工智能研究杂志53:375–438.
- McGovern, A., 和 Barto, A. G. 2001. 使用多样密度在强化学习中自动发现子目标。在 *ICML*, 361–368.
- Menache, I.; Mannor, S.; 和 Shimkin, N. 2002. Q-cut - 强化学习中子目标的动态发现。在 *ECML*, 295–306.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; 和 Riedmiller, M. A. 2013. 使用深度强化学习玩雅达利游戏。 *CoRR*abs/1312.5602.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T. P.; Harley, T.; Silver, D.; 和 Kavukcuoglu, K. 2016. 深度强化学习的异步方法。在 *ICML*.
- Niekum, S. 2013. 基于非结构化演示的语义基础学习。 博士论文, 马萨诸塞大学, 阿默斯特.
- Precup, D. 2000. 强化学习中的时间抽象。 博士论文, 马萨诸塞大学, 阿默斯特.
- Puterman, M. L. 1994. 马尔可夫决策过程：离散随机动态规划。 约翰·威利父子出版公司.
- Silver, D., 和 Ciosek, K. 2012. 使用最优选项模型的组合规划。在 *ICML*.
- Sorg, J., 和 Singh, S. P. 2010. 线性选项。在 *AAMAS*, 31–38.
- Stolle, M., 和 Precup, D. 2002. 强化学习中的学习选项。在 抽象、重构与近似, 第五届国际研讨会, *SARA* 论文集, 212–223.
- Sutton, R. S.; 和 Singh, S. P. 1999. 在MDP和半MDP之间：强化学习中时间抽象的框架。 人工智能112(1-2):181–211.
- Sutton, R. S. 1984. 强化学习中的时间信用分配。 博士论文. Thomas, P. 2014. 自然演员-评论家算法中的偏差。在 *ICML*, 441–448.
- Vezhnevets, A. S.; Mnih, V.; Agapiou, J.; Osindero, S.; Vinyals, O.; 和 Kavukcuoglu, K. 2016. 用于学习宏动作的策略性注意力写入器。在 *NIPS 29*.